CoopPilot — Stage 2 Implementation Report

EECS 3311 Software Design · Fall 2026 · AI–Human Collaborative Implementation

Sina Akhavan Safar · solo project · github.com/Box-OS/eecs3311-cooppilot

1. Project overview and implementation status
2. Feature-to-implementation mapping
   - F01 — Experience Vault and Resume Import (D)
   - F02 — Job Posting Import and Analysis (H)
   - F03 — Fit Analysis and Skill-Gap Detection (H)
   - F04 — ATS Keyword Coverage Scan (D)
   - F05 — Agentic Resume Tailoring (AI)
   - F06 — Clarifying Questions and Vault Learning (H)
   - F07 — One-Page Fitting (Estimate and Condense) (H)
   - F08 — Natural-Language Revision with Version History (H)
   - F09 — Cover Letter Drafting (AI)
   - F10 — Resume Library, Reuse and Export (D)
   - F11 — Application Tracking with Status Lifecycle (D)
   - F12 — Progress Dashboard and Goals (D)
3. Implemented design patterns
   - 3.1 Strategy — interchangeable condense algorithms
   - 3.2 Command — undoable resume edits
   - 3.3 Adapter — provider-independent LLM access
   - 3.4 Observer — live agent progress and dashboard updates
   - 3.5 State — application lifecycle
   - 3.6 Facade — one entry point for both UIs
   - 3.7 Template Method — single-shot LLM tasks
   - 3.8 Updated UML diagrams
4. Changes and rationale
   - 4.1 Consolidated record (Original design → Implemented change → Reason → Impact)
   - 4.2 Modified features (Section 3 items 1–5)
   - 4.3 Modified patterns (Section 3 items 6–10)
5. AI tools and collaboration log
   - 5.1 Tools and models
   - 5.2 How the collaboration worked
   - 5.3 Collaboration log (representative interactions)
6. Development and verification summary
   - 6.1 Weekly commit summary
   - 6.2 Verification performed
   - 6.3 Important defects found and corrected
   - 6.4 Remaining limitations
7. Demonstration index

1. Project overview and implementation status

CoopPilot is an AI agent that helps a co-op student apply for jobs. The student keeps their real experience in an *experience vault* (imported from their LaTeX resume and extended by hand). For each job posting, an LLM extracts the requirements and keywords, deterministic code scores the student's fit, and a tool-using agent builds a **one-page, tailored LaTeX resume** from the student's own material, asking the student when a fact is missing instead of guessing. The student can then revise the resume in plain English (with undo/redo), draft a cover letter, export the files for Overleaf, and track every application through its lifecycle on a progress dashboard.

Users. University co-op students (the author first) who apply to many postings each term and need each resume to fit the posting, stay truthful, and fit on one page.

AI / LLM integration. Anthropic Claude (default model `claude-haiku-4-5`) through the Messages API with native tool use, behind the `LLMClient` interface. The LLM is used in two ways:

- **Agent (F05–F07):** a reason–act–observe loop (`AgentController`) in which the model chooses among 9 tools (find similar resume, search vault, keyword scan, compose, estimate page, condense, ask user, save). This covers the required agent behaviours: *planning and multi-step execution* (the loop, up to 15 steps), *tool use* (validated by `ToolRegistry`), *retrieval* (vault and resume library search), *memory* (answers are filed in the vault and never asked again), and *decision making*

(reuse vs. rewrite, what to cut to fit one page).
- **Single-shot tasks (F02, F08, F09, smart F07):** posting analysis, natural-language revision, cover letters and smart condensing, each through the `LlmTask<T>` template, which validates the reply and retries with the problems listed.

Deterministic Java code enforces every rule the LLM could break: only facts from the vault (new numbers in rewritten text are rejected by `FactGuard`), reverse-chronological Experience, no bold words in Education/Skills, one page maximum, and `CV_Company_Role` / `CL_Company_Role` naming.

Current working functionality. All 12 Stage 1 features are implemented and reachable from **both** interfaces:

- **Web GUI** (Spring Boot 3 + Thymeleaf, `mvn spring-boot:run`, http://localhost:8080): Vault, Postings, live Tailoring page (agent steps streamed over Server-Sent Events, questions answered in the page), Resumes (revise/undo/redo, one-page meter and condense, cover letter, `.tex` download), Tracker, Dashboard.
- **CLI** (picocli, `java -jar target/cooppilot-0.1.0.jar --cli <command>`): a command for every feature, `--json` output for scripting (used by the Stage 3 KUMA tests), and an interactive `shell` in which undo/redo history persists between commands.
- **Storage:** SQLite (`cooppilot.db`, git-ignored). **Build:** Maven (wrapper included: `mvnw`/`mvnw.cmd`), Java 21; runs on Windows, macOS and Linux. **Tests:** 108 automated tests (JUnit 5, Spring Boot Test, MockMvc) pass.
- **Real use:** the agent was run end-to-end against Claude Haiku 4.5, and the author's real vault (65 entries) and resume library (50 resumes) were imported from his earlier tracker through the JSON backup import.

Limitations and incomplete functionality (honest list).

1. **Real-LLM behaviour is verified by manual runs, not by automated tests.** Automated tests drive the agent with a scripted `MockLlmClient`; the real model's tool use was checked manually (Section 6). Systematic agent evaluation is Stage 3 (KUMA).
2. **Only invented numbers are blocked deterministically.** `FactGuard` rejects new numbers, but a reworded bullet can still add small non-numeric embellishments (in one real run the model added "algorithms" to a tutoring bullet). The student reviews the "What changed" diff; deeper fact checking is Stage 3 work.
3. **Undo/redo history lives in memory**: it is lost when the application restarts, and in the CLI it only spans one process (hence the `shell` command).
4. **The page estimate is a heuristic** (line-cost model calibrated on 50 real one-page resumes), not a compiled PDF; the in-app preview approximates the layout and the student compiles the `.tex` in Overleaf.
5. **Single user, no authentication**; the app is meant to run locally.
6. Not implemented from the Stage 1 design: only the `OpenAiCompatibleAdapter` (a second LLM provider); Claude and the offline mock implement `LLMClient`. See C07.
7. Without an API key the app runs with the mock LLM: deterministic features work fully, LLM features return a fixed "no key" reply.

2. Feature-to-implementation mapping

Every Stage 1 feature (F01–F12) is accounted for below. No feature was removed or replaced; several were modified (details in Section 4). There are no new features beyond F01–F12; the CLI `shell` and `--json` mode are interface additions, not features. Type: **D** deterministic, **AI** LLM-based, **H** hybrid. Package prefix `ca.yorku.cooppilot` is omitted. Video links are in Section 7.

F01 — Experience Vault and Resume Import (D)

Field	Entry
Stage 1 reference	F01, UC01, SD08
Status	Implemented + extended (JSON backup import; see C21)
Classes / files	`service.LatexResumeParser` (splits a Jake's-template `.tex` into blocks), `service.LatexText` (brace-balanced argument reader, LaTeX→text, date ranges), `service.VaultService` (vault rules: dedupe, search, tag filter, skills, import), `service.BackupImporter` (JSON backup of vault entries and resumes), `vault-edit.html` (edit an entry), `domain.VaultEntry/EntryKind/EntryUse`, `domain.ResumeBlock/Resume/Section`, `persistence.SqliteVaultRepository`, `persistence.SqliteResumeRepository`, `app.CoopPilotFacade`, `ui.web.VaultController` + `vault.html`, `ui.cli.CliCommands.VaultCmd`
Key methods	`LatexResumeParser.parse(latex)`, `VaultService.importFromResume(parse)`, `VaultService.addEntry(e)`,

	`VaultService.search(q, tags)`, `CoopPilotFacade.importResume(latex, name)`, `importBackup(json)`, `updateVaultEntry/deleteVaultEntry`
Design patterns	Facade; Repository (not counted)
Video	V01

Collaboration. The Vault page uploads the `.tex` file (CLI: `vault import FILE`) → `VaultController` calls `CoopPilotFacade.importResume` → `LatexResumeParser` finds each `\section`, reads `\resumeSubheading` (detecting the education vs. experience argument order), `\resumeProjectHeading` (tech stack → tags), `\resumeItem` bullets and skill lines, and returns blocks plus warnings → the facade saves an `ORIGINAL` resume and `VaultService` creates one vault entry per experience and one skill entry per skill, merging duplicates → the page shows counts and warnings. Re-importing the same resume adds nothing. Entries can be edited (✎) or deleted (×), filtered by clicking a tag, and a duplicate added by hand (e.g. the same skill with different case or spacing) is refused with a message. **Import backup** (Vault page or `vault import-backup FILE`) loads a JSON file of vault entries and resumes, so a student can start from an existing vault; it was used to import the author's real data from his earlier tracker.

F02 — Job Posting Import and Analysis (H)

Field	Entry
Stage 1 reference	F02, UC02, SD03
Status	Implemented
Classes / files	`llm.PostingAnalysisTask` (prompt, JSON parsing, validation, soft-skill rule), `llm.LlmTask` (retry template), `llm.JsonReply` (extracts JSON from chatty replies), `domain.Posting/PostingAnalysis/Keyword`, `persistence.SqlitePostingRepository`, `CoopPilotFacade`, `ui.web.PostingController` + `postings.html/posting.html`, `CliCommands.PostingCmd`
Key methods	`CoopPilotFacade.importPosting(text, company, role)`, `CoopPilotFacade.analyzePosting(id)`, `CoopPilotFacade.saveAnalysis(...)`, `LlmTask.run()`, `PostingAnalysisTask.parse/validate`, `PostingAnalysisTask.softSkillFocus(mustHaves)`
Design patterns	Template Method, Adapter, Facade
Video	V02

Collaboration. The student pastes a posting (CLI: `posting add FILE --analyze`) → the facade saves it → `PostingAnalysisTask.run()` (template) builds a strict-JSON prompt, calls `LLMClient`, parses must-haves, nice-to-haves, keywords with aliases and required documents, and validates (keywords non-empty, requirements present); invalid replies are retried with the problems listed. The **soft-skill focus** flag is then recomputed deterministically (2+ or 40%+ of must-haves are soft skills). Blank company/role/term/deadline are filled from the analysis. The student can **edit the analysis** (company, role, term, deadline, requirements, keywords as `term | aliases | must`, documents), or fill it in by hand when the LLM analysis fails (UC02 3a).

F03 — Fit Analysis and Skill-Gap Detection (H)

Field	Entry
Stage 1 reference	F03, UC03, SD03
Status	Implemented (TWEAK threshold added, see C-F03)
Classes / files	`service.FitScorer` (weighted score, gaps, strategy thresholds), `service.ResumeLibrary.findClosest` (closest saved resume), `domain.FitReport/ResumeMatch/TailorStrategy`, `CoopPilotFacade.analyzeFit`, `posting.html`, `CliCommands.FitCmd`
Key methods	`FitScorer.score(analysis, skills, closest)`, `ResumeLibrary.findClosest(analysis)`, `FitScorer.strategy(closest)`
Design patterns	Facade
Video	V03

Collaboration. After analysis, the posting page (CLI: `fit POSTING_ID`) calls `CoopPilotFacade.analyzeFit` → `FitScorer` matches each keyword (must-haves weigh 2) against the vault's skills and lists the gaps, must-haves first → `ResumeLibrary.findClosest` scores every saved resume that has no `[placeholders]` → the strategy is **REUSE** at ≥ 75% match, **TWEAK** at ≥ 45%, else **GENERATE**. The LLM's part is the analysis from F02; the scoring is deterministic.

F04 — ATS Keyword Coverage Scan (D)

Field	Entry
Stage 1 reference	F04, UC03, SD03
Status	Implemented
Classes / files	`service.KeywordScanner`, `service.ResumeText` (plain-text views of a resume; "evidence" text excludes the skills line), `domain.Keyword.appearsIn` (word-boundary matching: Java ≠ JavaScript, C++ handled), `domain.KeywordReport`, `agent.tools.KeywordScanTool`, chips on `posting.html` and `resume.html`, `CliCommands.ScanCmd`
Key methods	`KeywordScanner.scan(resume, analysis)`, `Keyword.appearsIn(text)`, `KeywordReport.coverage()`
Design patterns	N/A (used as an agent tool)
Video	V04

Collaboration. For a chosen resume, `KeywordScanner` strips LaTeX, then classifies each posting keyword as **backed** (in experience/project text), **thin** (only in the skills line), **known but not worded** (the vault supports it) or **missing**. The posting page and the resume page show coloured chips (on the resume page they refresh after every edit, for any chosen posting); the agent calls the same scanner through `keyword_scan`.

F05 — Agentic Resume Tailoring (AI)

Field	Entry
Stage 1 reference	F05, UC04, SD01, SD09
Status	Implemented (direct HTTP adapter instead of the SDK; richer `Message`; autonomy nudges; see C01, C02, C20)
Classes / files	`agent.AgentController` (loop, step limit, LLM retry), `agent.AgentSession` (conversation, draft, listeners), `agent.PromptBuilder` (goal + hard rules), `agent.tools.ToolRegistry` (argument validation), tools `FindSimilarResumeTool`, `SearchVaultTool`, `KeywordScanTool`, `ComposeResumeTool`, `EstimatePageTool`, `CondenseTool`, `AskUserTool`, `SaveResumeTool`; `service.LatexAssembler`, `service.FactGuard`, `service.ExportService.fileName`; `llm.ClaudeAdapter`; `ui.web.AgentSessionManager`, `SseProgressPublisher`, `TailorController` + `tailor.html`; `CliCommands.TailorCmd`
Key methods	`CoopPilotFacade.tailorResume(postingId, q, listener)`, `AgentController.run/loop/completeWithRetry`, `ToolRegistry.execute/validateArgs`, `ComposeResumeTool.execute`, `LatexAssembler.assemble`, `SaveResumeTool.execute`
Design patterns	Observer, Adapter, Facade (+ Strategy through `CondenseTool`)
Video	V05

Collaboration. "Tailor with the agent" (CLI: `tailor POSTING_ID`) → `AgentSessionManager` starts the run on a background thread → `AgentController` sends the system prompt (goal + rules) and tool schemas to Claude → each tool call the model makes is validated by `ToolRegistry` (unknown tools, missing or mistyped arguments and tool exceptions go back to the model as errors) and executed; results return to the conversation → typical plan: `find_similar_resume` → `search_vault` → `compose_resume` (keep/reorder blocks, front-load skills, reword bullets; rewrites with invented numbers are rejected) → `keyword_scan` → `estimate_page` → `condense` if needed → `save_resume` (refused over one page; named `CV_Company_Role`). `SseProgressPublisher` (observer) streams every step to the browser. The run ends when `save_resume` has succeeded and the model stops (`COMPLETED`), at 15 steps (`STEP_LIMIT`), or on a non-retryable LLM error (`FAILED`); 429/5xx errors are retried with backoff. If the model stops to chat without saving (seen in the first real run), the loop tells it to finish and save, at most twice. When a run stops early, the partial draft is kept as `CV_Company_Role_Draft` (UC04 5a).

F06 — Clarifying Questions and Vault Learning (H)

Field	Entry
Stage 1 reference	F06, UC06, SD02
Status	Modified (answers filed as fact lines; repeats answered from the vault; see C-F06)

Classes / files	`agent.tools.AskUserTool`, `agent.QuestionHandler` (interface), `ui.web.WebQuestionHandler` (waits ≤ 5 min for the browser's answer), CLI stdin handler in `CliCommands.TailorCmd`, `service.VaultService.fileAnswer/knownAnswer`
Key methods	`AskUserTool.execute`, `QuestionHandler.ask(question)`, `AgentSessionManager.answer(runId, answer)`, `VaultService.fileAnswer(q, a, topic, kind)`, `VaultService.knownAnswer(q)`
Design patterns	Interface polymorphism (`QuestionHandler`)
Video	V06

Collaboration. When a non-numeric fact is missing, the model calls `ask_user` → `AskUserTool` first checks the vault for an earlier answer to the same question → otherwise `QuestionHandler.ask` shows the question (web: an SSE `question` event and an answer box; CLI: a `?` prompt) → the answer is added to the vault entry it is about (or a new entry) → the tool result goes back to the model. An empty answer or a timeout tells the model not to guess.

F07 — One-Page Fitting (Estimate and Condense) (H)

Field	Entry
Stage 1 reference	F07, UC05, SD01
Status	Implemented (+ undoable `CondenseCommand`; see C03, C19)
Classes / files	`service.PageEstimator`, `service.CondenseStrategy` (interface), `service.RuleBasedCondense`, `service.LlmCondense` + `llm.SmartCondenseTask`, `service.edit.CondenseCommand`, `agent.tools.EstimatePageTool/CondenseTool`, `AppConfig.condenseStrategies`, `resume.html` (page meter, Quick/Smart condense), `CliCommands.CondenseCmd`
Key methods	`PageEstimator.estimate(resume)`, `CondenseStrategy.condense(resume, analysis, pages)`, `CoopPilotFacade.condenseResume(resumeId, postingId, mode)`, `ResumeEditor.condense`
Design patterns	Strategy, Command, Template Method (smart)
Video	V07

Collaboration. `PageEstimator` sums each block's heading and wrapped-bullet lines into a fraction of a page (shown as a meter, red when over 100%). **Quick condense** (`RuleBasedCondense`, free) removes the least relevant content in a fixed order: irrelevant projects (keeping 2) → projects, then jobs, to 2 bullets → weakest extra project → projects, then jobs, to 1 bullet. Jobs give way last and one project always stays. **Smart condense** (`LlmCondense`) asks the model what to cut or shorten (validated: no unknown bullets, no new numbers); if its plan keeps failing validation it falls back to Quick condense (UC05 2a), and it finishes with the rules if still over. Condensing aims at ≤ 98% of a page; saving requires ≤ 100%. From the GUI/CLI the condense runs as a `CondenseCommand`, so it can be undone; inside the agent it runs through `CondenseTool`.

F08 — Natural-Language Revision with Version History (H)

Field	Entry
Stage 1 reference	F08, UC07, SD04
Status	Implemented
Classes / files	`service.edit.ResumeEditor` (one `RevisionHistory` per resume; reassembles and saves after each step; records the last diff), `service.edit.ResumeDiff` ("What changed"), `RevisionHistory` (invoker), `ResumeEditCommand` + `AiRevisionCommand`, `ReplaceBulletCommand`, `RemoveBlockCommand`, `CondenseCommand`; `llm.RevisionTask`; `service.FactGuard`; `resume.html`; `CliCommands.ReviseCmd/EditBulletCmd/UndoCmd/RedoCmd`
Key methods	`CoopPilotFacade.reviseResume/replaceBullet/removeBlock/undo/redo/revisionLog`, `RevisionHistory.run/undo/redo`, `AiRevisionCommand.execute/undo`
Design patterns	Command, Template Method
Video	V08

Collaboration. The student types a request (CLI: `revise ID "..."`) → `ResumeEditor` creates an `AiRevisionCommand` and runs it through the resume's `RevisionHistory` → the command snapshots the blocks and runs `RevisionTask` (bullet rewrites/removals as JSON;

validation rejects unknown bullets and numbers not in the vault or resume) → the LaTeX is reassembled and saved. Undo restores the snapshot; redo reapplies the stored result **without calling the LLM again**. A new edit clears the redo branch. After every edit, undo or redo the page shows **What changed** (added, removed and reworded bullets, old text struck through, new text in green). Bullets can also be edited by hand (✎) and entries removed (×), both undoable.

F09 — Cover Letter Drafting (AI)

Field	Entry
Stage 1 reference	F09, UC08, SD05
Status	Implemented (validation window 180–450 words; see C12)
Classes / files	`llm.CoverLetterTask`, `persistence.CoverLetterRepository`, `domain.CoverLetter`, `CoopPilotFacade.writeCoverLetter`, `ui.web.ResumeController` + `letter.html`, `CliCommands.LetterCmd`
Key methods	`CoopPilotFacade.draftCoverLetter(postingId, resumeId)`, `saveCoverLetter(name, body)`, `CoverLetterTask.buildRequest/parse/validate`
Design patterns	Template Method
Video	V09

Collaboration. From a resume page (CLI: `letter POSTING_ID RESUME_ID`) → the facade collects only vault entries allowed in letters (`ALL`, `LETTERS_ONLY`; never `CONTEXT_ONLY`) → `CoverLetterTask` asks for a 250–350-word plain-text letter that states the posting's term and connects 2–3 real experiences to the must-haves → validation rejects bad length, letters that don't name the company, `[placeholders]` and invented numbers, with retries → saved as `CL_Company_Role` and shown in an editable text area (Save, Download .txt). If the draft still fails validation after the retries, it is saved anyway and shown with the problems listed (UC08 3a).

F10 — Resume Library, Reuse and Export (D)

Field	Entry
Stage 1 reference	F10, UC09, SD06
Status	Modified (match formula; see C-F10)
Classes / files	`service.ResumeLibrary` (closest match, `reuseAs`, grouping, rename, family, delete), `service.ExportService` (naming, `.tex`/`.txt` export with sanitized names), `resumes.html`/`resume.html`/`posting.html` ("Use as-is", download), `CliCommands.LibraryCmd/ReuseCmd/ExportCmd`
Key methods	`CoopPilotFacade.reuseResume(postingId, resumeId)`, `ResumeLibrary.reuseAs`, `ResumeLibrary.groupByFamily`, `ExportService.fileName/writeTex/writeLetter`
Design patterns	Facade
Video	V10

Collaboration. When F03 recommends REUSE, "Use as-is" copies the saved resume unchanged as `CV_Company_Role` (the original is kept; resumes with placeholders are refused; repeating replaces the copy). The library groups resumes by role family; any resume downloads as `.tex` (GUI) or is written to `exports/` (CLI `export`), with names sanitized so they cannot leave that folder.

F11 — Application Tracking with Status Lifecycle (D)

Field	Entry
Stage 1 reference	F11, UC10, SD07
Status	Modified (state objects as constants; exception type; see C-State)
Classes / files	`tracking.Application` (context), `tracking.ApplicationState` (state; seven concrete states), `tracking.Status`, `tracking.StatusEvent`, `tracking.ApplicationTracker` (subject), `tracking.SqliteApplicationRepository`, `ui.web.TrackerController` + `tracker.html`, `CliCommands.AppCmd`
Key methods	`CoopPilotFacade.logApplication/changeStatus`, `ApplicationTracker.transition(id, target, note)`, `Application.transitionTo`, `ApplicationState.moveTo/onEnter/allowedNext`

Design patterns	State, Observer
Video	V11

Collaboration. An application starts in **To apply** → moving it (GUI dropdown shows only legal next states; CLI `app status ID STATUS`) calls `ApplicationTracker.transition` → `Application` delegates to its current state, which refuses illegal moves (e.g. Rejected → Offer) and runs entry actions (**Applied** stamps the date and a +10-day follow-up; final states clear it; **Ghosted** can still be revived by a late reply) → the history event is saved and listeners are notified. An application can also be logged straight from a resume page ("Log application"), linking that resume.

F12 — Progress Dashboard and Goals (D)

Field	Entry
Stage 1 reference	F12, UC11, SD07
Status	Implemented (weekly chart = last 7 days)
Classes / files	`tracking.DashboardService` (observer with cached stats), `tracking.DashboardStats`, `tracking.Goals`, `settings` table, `ui.web.TrackerController.dashboard` + `dashboard.html`, `CliCommands.DashboardCmd/GoalsCmd`
Key methods	`DashboardService.stats()`, `DashboardService.compute(apps, goals, today)`, `DashboardService.streak(...)`, `CoopPilotFacade.dashboard/setGoals`
Design patterns	Observer
Video	V12

Collaboration. `DashboardService` registers with `ApplicationTracker` and drops its cached statistics whenever an application changes (or the goals change); `stats()` recomputes only then (and when the date changes): today and this week vs. goals (with "Goal met"), total applied, response rate, interviews, offers, follow-ups due, daily-goal streak and a last-7-days bar chart, plus the list of follow-ups due and an empty state when there are no applications. Goals are stored in SQLite and must be at least 1 (UC11 3a).

3. Implemented design patterns

Seven distinct patterns are implemented, matching the seven planned in Stage 1. MVC, Repository and dependency injection are also used but are not counted.

3.1 Strategy — interchangeable condense algorithms

- **Problem.** A resume can be fitted to one page cheaply with fixed rules or more cleverly with the LLM. The agent, the GUI buttons and the CLI must all trigger "condense" without knowing which algorithm runs.
- **Roles.** Strategy: `service.CondenseStrategy`. Concrete strategies: `service.RuleBasedCondense`, `service.LlmCondense`. Contexts: `agent.tools.CondenseTool` (agent) and `service.edit.CondenseCommand` (GUI/CLI). The strategies are registered by mode in `AppConfig.condenseStrategies()` (`Map<Mode, CondenseStrategy>`).
- **Files / methods.** `CondenseStrategy.condense(resume, analysis, pages)`, `lastReport()`; `CondenseTool.execute` and `CondenseCommand.execute` select by `Mode` and never branch on the algorithm.
- **Why useful.** Same signature, different cost and behaviour, chosen at run time (the agent's `mode` argument or the button pressed). `LlmCondense` even reuses `RuleBasedCondense` as its fallback. Adding a third algorithm means one class and one map entry.
- **Stage 1.** Matches. Change: the GUI/CLI context is `CondenseCommand` instead of the facade, so condensing is undoable (C03).

3.2 Command — undoable resume edits

- **Problem.** AI revisions, manual bullet edits, block removals and condensing all change a resume; the student must be able to undo and redo any of them in order, and an AI revision (expensive, non-repeatable) must be reversible exactly.
- **Roles.** Command: `service.edit.ResumeEditCommand`. Concrete commands: `AiRevisionCommand`, `ReplaceBulletCommand`, `RemoveBlockCommand`, `CondenseCommand`. Invoker: `RevisionHistory` (undo/redo stacks). Receiver: `domain.Resume`. Client: `ResumeEditor` (creates commands; one history per resume).
- **Files / methods.** `RevisionHistory.run/undo/redo/log`; each command's `execute()` stores what `undo()` needs (old text, removed block and position, or a before/after snapshot).
- **Why useful.** The history treats every edit uniformly; `AiRevisionCommand` calls the LLM only on the first `execute()` and redo

reapplies the stored result.
- **Stage 1.** Matches, with one added concrete command (`CondenseCommand`) and the client moved from the facade to `ResumeEditor` (C-Command).

3.3 Adapter — provider-independent LLM access

- **Problem.** The agent and the LLM tasks need one request/response format with tool calls, independent of any provider's wire format, and must be testable without the network.
- **Roles.** Target: `llm.LLMClient`. Adapter: `llm.ClaudeAdapter` (translates `LlmRequest`/`LlmResponse`/`ToolCall`/`Message` to and from the Anthropic Messages API JSON: `tool_use`/`tool_result` blocks, merged consecutive tool results, usage). Second implementation: `llm.MockLlmClient`. Clients: `AgentController`, `LlmTask` subclasses, `LlmCondense`.
- **Files / methods.** `ClaudeAdapter.complete`, `toProvider`, `fromProvider`; provider chosen in `AppConfig.llmClient` from `cooppilot.llm.provider`.
- **Why useful.** No provider type appears outside the adapter; switching models or providers touches one class; every agent test runs on the mock.
- **Stage 1.** Partly different: the adaptee is the HTTP API directly (not the Java SDK), and `OpenAiCompatibleAdapter` was not built (C01, C-Adapter).

3.4 Observer — live agent progress and dashboard updates

- **Problem.** (a) The browser and the CLI show each agent step live, but the agent core must not depend on either UI. (b) Dashboard statistics must stay correct whenever any application changes, without recomputing on every page view.
- **Roles.** (a) Subject `agent.AgentSession`; observer interface `agent.AgentEventListener`; concrete observers `ui.web.SseProgressPublisher` (SSE events to the browser) and the CLI's printer in `CliCommands.TailorCmd`. (b) Subject `tracking.ApplicationTracker`; observer interface `tracking.ApplicationListener`; concrete observer `tracking.DashboardService` (invalidates its cache).
- **Files / methods.** `AgentSession.addListener`, `notifyToolCalled/notifyToolResult/notifyFinished`; `ApplicationTracker.addListener`, `transition` → `onApplicationChanged`.
- **Why useful.** One-to-many notification with no compile-time dependency from the core on any UI; the dashboard is cheap to read.
- **Stage 1.** Matches; the CLI observer is an anonymous listener inside `TailorCmd` rather than `CliRunner` itself (C-Observer).

3.5 State — application lifecycle

- **Problem.** What an application can do next depends on its status, and entering a status has side effects (Applied schedules a follow-up).
- **Roles.** Context: `tracking.Application`. State: `tracking.ApplicationState` (abstract). Concrete states: the seven constants `TO_APPLY`, `APPLIED`, `ASSESSMENT`, `INTERVIEW`, `OFFER`, `REJECTED`, `GHOSTED`, each an anonymous subclass that defines its legal next statuses and overrides `onEnter` where needed. `tracking.Status` is the stored name and maps back to its state.
- **Files / methods.** `Application.transitionTo(target, today, note)` delegates to `ApplicationState.moveTo`, which checks `canMoveTo` and calls the new state's `onEnter`.
- **Why useful.** No status `switch` anywhere; the GUI menu is built from `allowedNext()`; adding a status touches one place.
- **Stage 1.** Same pattern; different form (stateless shared constants instead of seven top-level classes, `IllegalStateException` instead of a custom exception, transitions by target status instead of events; C-State).

3.6 Facade — one entry point for both UIs

- **Problem.** A user action such as "tailor a resume" involves the agent, vault, library, repositories, editor and tracker, and two UIs need identical capabilities.
- **Roles.** Facade: `app.CoopPilotFacade`. Subsystems: `VaultService`, `LatexResumeParser`, `ResumeLibrary`, `FitScorer`, `KeywordScanner`, `AgentController`, `ResumeEditor`, `ExportService`, `ApplicationTracker`, `DashboardService`, repositories. Clients: `VaultController`, `PostingController`, `ResumeController`, `TrackerController`, `AgentSessionManager`, `CliCommands`.
- **Why useful.** Controllers and CLI commands are thin and call only the facade, so the GUI and CLI cannot drift apart; the CLI's `--json` mode becomes a ready-made test entry point for Stage 3.
- **Stage 1.** Matches.

3.7 Template Method — single-shot LLM tasks

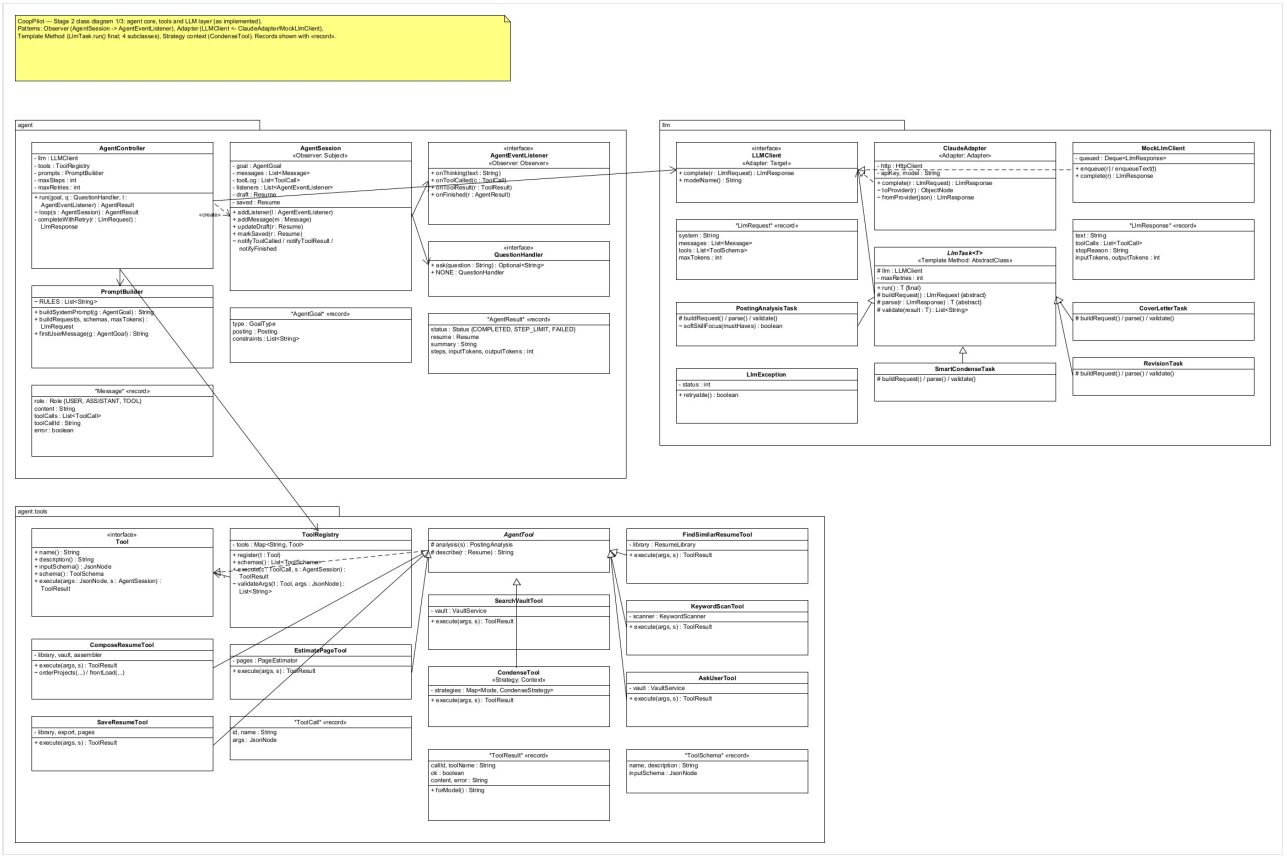
- **Problem.** Posting analysis, revisions, cover letters and smart condensing all build a request, call the LLM, parse, validate and retry on invalid output; only the prompt, parsing and checks differ.
- **Roles.** Abstract class `llm.LlmTask<T>` with `final run()`; concrete classes `PostingAnalysisTask`, `RevisionTask`, `CoverLetterTask`,

`SmartCondenseTask` overriding `buildRequest()`, `parse()` and the `validate()` hook.

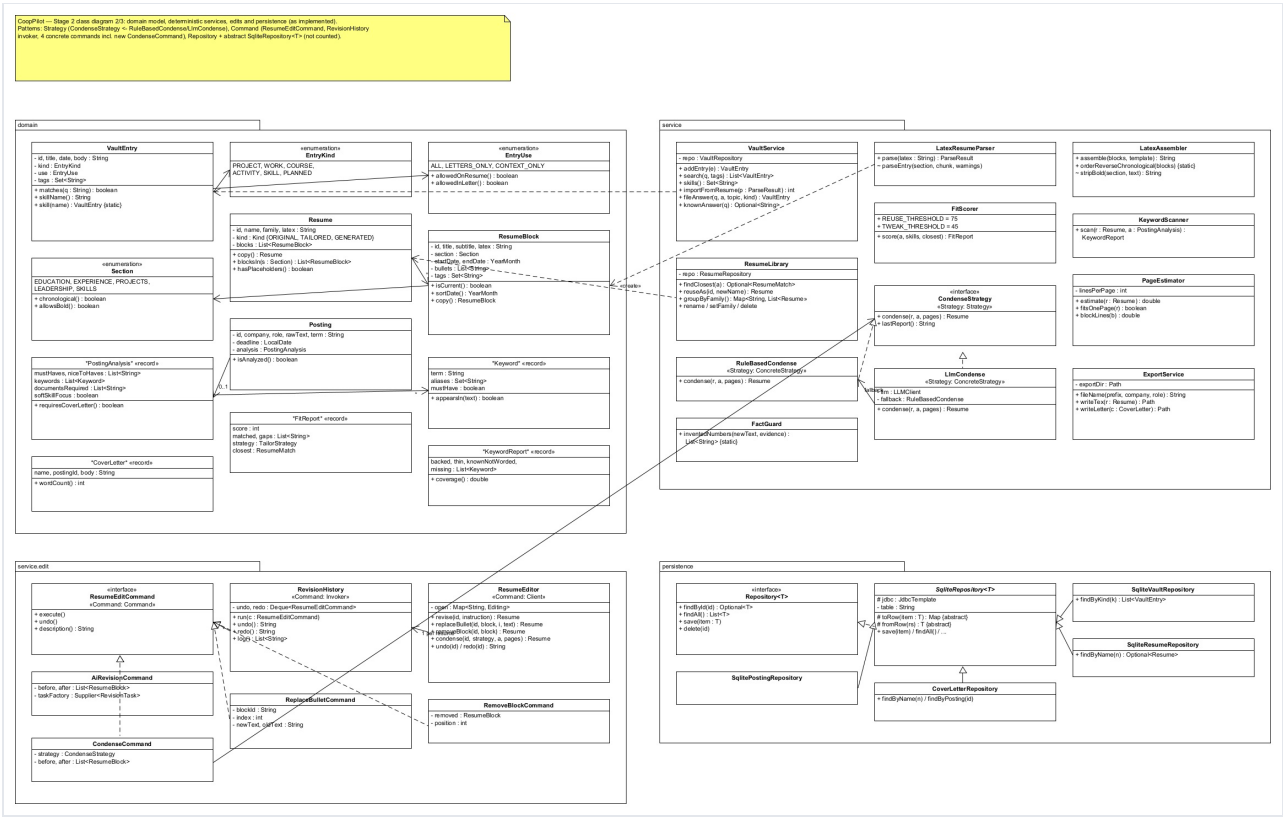
- **Why useful.** The retry-with-feedback policy, which guards against malformed or invented output, is written once and cannot be skipped (`run()` is final).
- **Stage 1.** Matches.

3.8 Updated UML diagrams

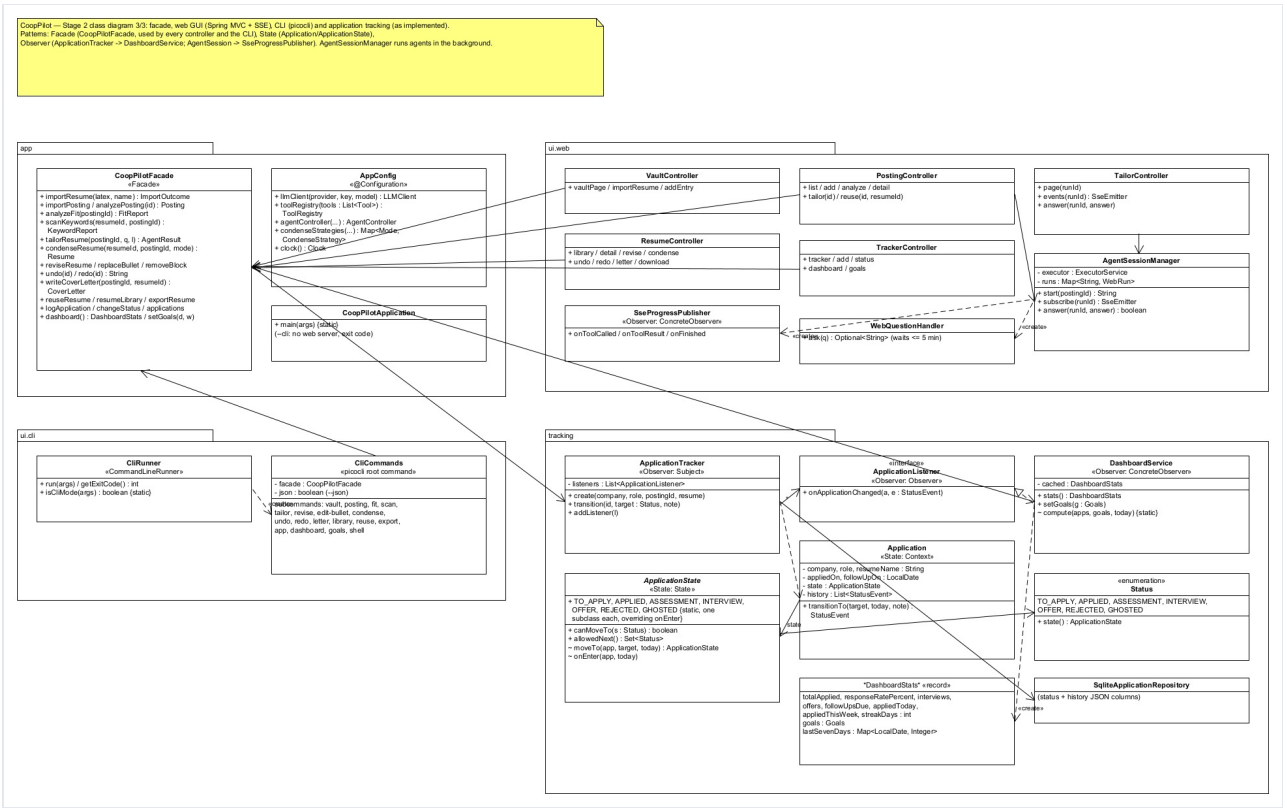
The class diagrams were redrawn for the implemented system in UMLet (sources in `docs/stage2/diagrams/*.uxf`, PNG exports beside them). Pattern roles are marked with stereotypes.



Class diagram 1 — agent core, tools and LLM layer



Class diagram 2 — domain, services, edit commands and persistence



Class diagram 3 — facade, web GUI, CLI and application tracking

Use-case and sequence diagrams. The Stage 1 use-case diagram and SD01–SD08 still describe the implemented interactions; the differences are at the method level and are listed in Section 4 (notably: SD02 answers are filed by `VaultService.fileAnswer` as a fact line; SD04 commands are created by `ResumeEditor`, not the facade; SD06 export writes

`exports/NAME.tex`; SD09 commands are `cooppilot --cli tailor POSTING_ID` and progress is printed by a listener in `TailorCmd`). No use case was added or removed.

4. Changes and rationale

4.1 Consolidated record (Original design → Implemented change → Reason → Impact)

ID	Original design	Implemented change	Reason	Impact
C01	`ClaudeAdapter` wraps the Anthropic Java SDK	Builds Messages API JSON with Jackson and sends it with `java.net.http.HttpClient`	One less dependency to version-match; translation is small and unit-testable offline	Adapter unchanged; `AnthropicClient` removed from the class diagram
C02	`Message(role, content)`	`Message(role, content, toolCalls, toolCallId, error)`, roles USER/ASSISTANT/TOOL	The loop must send tool calls and their results back to the model	`AgentSession.messages`, `LlmRequest` carry the full tool conversation
C03	`CondenseStrategy` with two strategies, context = facade	Adds `lastReport()`; `LlmCondense` falls back to rules; GUI/CLI condensing runs as `CondenseCommand`	Users and the agent must see what was cut; result must always fit; condensing should be undoable	Strategy unchanged; Command gains a 4th concrete command
C04	Facade owns one `RevisionHistory`	`ResumeEditor` keeps one history per resume and saves after each step	Several resumes can be edited independently	Facade delegates to `ResumeEditor` (C-Command)
C05	CLI `coop …` commands; `CliRunner` observes the agent	`cooppilot --cli …` via picocli `CliCommands`; progress printed by a listener in `TailorCmd`; new `shell`, `condense`, `--json`	Spring Boot runs the CLI in the same jar; undo history needs one process	Observer role moves; CLI covers all 12 features
C06	7 concrete state classes, `handle(event)`, custom exception	7 state constants (anonymous subclasses) in `ApplicationState`, `transitionTo(target)`, `IllegalStateException`	States carry no data, so shared instances suffice; status names map 1:1 to the stored enum	State pattern kept; fewer files (C-State)
C07	`OpenAiCompatibleAdapter`	Not implemented	Only Claude is used; the mock covers offline/test use	Adapter has one real adapter + mock
C08	F10 match = skill overlap 65% + role-family similarity 35%	Share of the posting keywords the student has that the resume shows (must-haves ×2)	No reliable role-family classifier yet; keyword evidence is what an ATS sees	Same thresholds (≥ 75% reuse)
C09	F03 thresholds: ≥ 75% reuse only	REUSE ≥ 75%, TWEAK ≥ 45%, else GENERATE	A strategy needed a defined boundary between tweak and generate	`FitScorer.TWEAK_THRESHOLD`
C10	F06 answer rewritten by the LLM into a clean vault entry; repeat questions answered from the session	Answer appended as a fact line to the matching entry (or a new entry); repeats answered from the vault	Deterministic filing cannot lose or alter facts; vault lookup works across sessions	`VaultService.fileAnswer/knownAnswer`
C12	F09 validation 250–400 words	Prompt asks for 250–350, validation accepts 180–450	A strict range caused needless retries for letters that were fine	`CoverLetterTask.validate`
C14	F01 duplicate skill rejected with a message	Refused with a message when added by hand; during imports duplicates are merged (tags kept) silently	Re-importing the same resume or backup should be harmless	`VaultService.addEntry`, `VaultController.addEntry`
C15	F05 LLM failure → retry with backoff	Implemented: 429/5xx/network retried twice with doubling waits; 4xx fail at once	Matches Stage 1 (added after the first implementation review)	`AgentController.completeWithRetry`

C16	— (new class)	`FactGuard` shared deterministic check for invented numbers	Same rule needed by compose, revision and cover letters	Used by 3 classes
C17	`AgentSessionManager` runs the agent	Also buffers events so late or reconnecting pages see every step	The page subscribes after the run starts	SSE robust to timing
C18	Dates from the system clock	`Clock` bean injected into tracker and dashboard	Follow-up and streak rules must be testable	Test-only impact
C19	F07 page estimate (no calibration specified)	`PageEstimator` calibrated to 63 lines per page; condense target 98%, save limit 100%	With the author's 50 real one-page resumes imported, all estimated at 102–113%; now 83–98%	The agent no longer condenses or refuses to save good resumes
C20	F05 loop ends when the model stops calling tools	Rules state the agent works autonomously; if the model stops without saving, the loop nudges it to finish and save (max 2)	First real run with Claude Haiku 4.5 ended with the model asking "Would you like me to…?" in chat, so nothing was saved	`PromptBuilder.nudge`, `AgentController.MAX_NUDGES`
C21	— (new, extends F01)	JSON backup import of vault entries and resumes (`BackupImporter`; Vault page and `vault import-backup`)	Lets a student start from an existing vault; used to import the author's real data from his earlier tracker	No new use case; UC01 gains an alternative input
C22	UC04 5a partial draft shown	Partial draft saved as `CV_Company_Role_Draft` (`DraftSaver`); `AgentResult.resumeSaved`	The draft is otherwise lost when the run stops; a saved draft can be opened and finished	`AgentController.finish`, `AppConfig`
C23	Resume preview (Stage 1 mentions a preview)	`ResumePreview` renders a Jake's-template-like HTML page from the blocks (all text HTML-escaped)	LaTeX is compiled outside the app; the student needs a readable view in it	`resume.html`, `LatexText.toHtml`

4.2 Modified features (Section 3 items 1–5)

No feature was replaced or removed. F01 was extended with a JSON backup import (C21). The modified features are:

Feature	1. Original	2. Implemented behaviour	3. What changed and why	4. User need	5. Affected use cases / classes / methods / patterns
C-F01	F01	Adds JSON backup import of vault entries and resumes	Students (and the author) already keep their experience elsewhere	Faster start with a complete vault	UC01; `BackupImporter`, `VaultController.importBackup`, `CliCommands.VaultCmd.ImportBackup`
C-F03	F03	Adds TWEAK ≥ 45%	Needed a defined middle band	Same: score, gaps, strategy	UC03; `FitScorer.strategy`
C-F06	F06	Fact lines + vault lookup	Deterministic, cross-session memory	Agent still never asks twice	UC06; `AskUserTool`, `VaultService.fileAnswer/knownAnswer`
C-F09	F09	180–450 word validation window	Fewer needless retries	Letter still grounded, named, editable and exportable	UC08; `CoverLetterTask.validate`
C-F10	F10	Keyword-evidence match %	No role-family classifier	Reuse suggestion still at ≥ 75%	UC09; `ResumeLibrary.findClosest/matchPercent`

4.3 Modified patterns (Section 3 items 6–10)

Pattern	6. Original	7. Implemented	8. Classes and roles	9. Problem addressed	10. Why appropriate / architectural effect
C-State	State with 7 classes	State with 7 shared	`Application` context,	Legal moves and	States hold no data,

	and events	state objects, transitions by target	`ApplicationState` state, 7 constants concrete states, `Status` mapping	entry actions per status	so singletons avoid allocation and keep all rules in one file; GUI reads `allowedNext()`
C-Command	Command with facade as client, 3 commands	Command with `ResumeEditor` as client, 4 commands	`ResumeEditCommand`, `RevisionHistory` invoker, `ResumeEditor` client, `Resume` receiver, `CondenseCommand` new	Uniform undo/redo for all edits	One history per resume; condensing joins the same undo stack; facade stays thin
C-Adapter	2 adapters over SDK/HTTP	1 adapter over HTTP + mock	`LLMClient` target, `ClaudeAdapter` adapter, `MockLlmClient`	Provider independence and offline tests	Same isolation with fewer dependencies
C-Observer	`CliRunner` as CLI observer	Listener inside `CliCommands.TailorCmd`	`AgentSession` subject, `AgentEventListener`, `SseProgressPublisher`, CLI listener	Live progress without UI dependencies	The picocli command owns its output stream, so it owns the listener

5. AI tools and collaboration log

5.1 Tools and models

Tool / platform	LLM / model	Used for (features, classes, tasks)	Total token cost	Money cost
Claude (claude.ai app, agentic workspace session with shell, file and GitHub access)	Claude Opus 5.5 (`claude-opus-5-5`)	Stage 2 planning; all implementation commits (skeleton, domain, persistence, LLM layer, F01–F12, CLI, GUI); the 86 automated tests; UMLet class-diagram generation; drafting this report, the change log and the demo script	≈ 38.6 M tokens processed in the session up to Nov 12, 2026: 36.7 M cache reads, 1.6 M cache writes, 0.27 M output (≈ 8.3 M input-token-equivalents when cache reads are weighted 0.1×, cache writes 2× and output 5×)	No per-token charge: covered by the author's **Claude Pro** subscription, **CA$31.64 per month**. All Stage 2 work (Oct 21 – Nov 15) falls within about one month of the subscription.
Anthropic API (runtime LLM inside CoopPilot, via `ClaudeAdapter`)	Claude Haiku 4.5 (`claude-haiku-4-5`)	Posting analysis (F02), the tailoring agent (F05–F07), revisions (F08), cover letters (F09), smart condense (F07) during manual testing and the demo recordings	**[TODO: total tokens from console.anthropic.com → Usage]**	**[TODO: US$ from the Console]**

Token counts for Claude come from the session transcript's usage records. API keys are kept in the git-ignored `application-local.properties` and appear in no commit, recording or report.

5.2 How the collaboration worked

The author set the direction and the rules; Claude wrote most of the code and documents; the author ran, checked and demonstrated the result. CoopPilot is a Java redesign of the author's own earlier browser-based prototype (a personal job-application tracker), so most product rules come from his real use: one-page resumes, reverse-chronological experience, no bold words in Education/Skills, the `CV_Company_Role` / `CL_Company_Role` naming, never inventing facts or numbers, and asking the student instead of guessing. The Stage 1 design (written with AI assistance in an earlier session) was the specification given to the AI for each step.

5.3 Collaboration log (representative interactions)

#	Task	Input (from the author)	AI contribution	Human contribution	Outcome
1	Stage 2 plan (Oct 21)	"How do I keep developing my tracker and still deliver Stage 2 with minimal extra work?" plus the	Mapped 11 of the 12 Stage 1 features to the prototype's existing logic; proposed porting the behaviour (not the code) to Java, keeping the Stage 1 structure, logging deviations as they happen, and a design freeze	Chose the approach; decided the Stage 1 design may change and must be	Plan adopted; weekly scope fixed

		Stage 2 handout	before the videos	documented; supplied the handout	
2	Week-1 skeleton and F01 (Oct 26)	“Make the 5 commits”; stack fixed in Stage 1 (Java 21, Spring Boot + Thymeleaf, no JavaFX, SQLite, picocli)	`pom.xml`, git-ignored key config with a placeholder example, domain model, `Repository<T>`/`SqliteRepository<T>`, `LLMClient`/`ClaudeAdapter`/mock, `LatexResumeParser`, `VaultService`, vault page, 28 tests	Set the constraints; required the key never be committed; asked that commits carry his name only (AI use is disclosed here instead)	6 commits in period 1, all tests passing
3	Adapter design decision	—	Proposed calling the Messages API over `HttpClient` instead of the SDK and documented it as C01	Accepted the deviation	Translation fully unit-tested offline (`ClaudeAdapterTest`)
4	Agent and F02–F07 (Nov 2)	“Queue in the next 5 commits”; rules from his prototype (one page, chronology, no bold, naming, no invented facts)	`LlmTask` template, posting analysis, fit/keyword scan, agent loop with `ToolRegistry` validation, 8 tools, `LatexAssembler`, condense strategies, `ask_user` with vault learning; end-to-end agent test with scripted LLM decisions	Supplied the rules the code had to enforce; decided the agent must ask instead of guessing	6 commits in period 2; tests caught a real naming bug (“Royal Bank of Canada” → `RBOC` instead of `RBC`), fixed before commit
5	F08–F12, CLI and GUI (Nov 9)	“Do the 6 commits”	State-pattern tracker, observing dashboard, Command-based revision with undo/redo, cover letters, library/export, picocli CLI with `--json` and `shell`, web pages with SSE progress; MockMvc walkthrough; screenshots of the running app checked	Chose the scope and order; reviewed the result **[TODO: what you checked or changed]**	6 commits in period 3; defects found and fixed (Section 6.3)
6	Design-vs-code review and report (Nov 12)	“Do the reports”	Compared the code to every Stage 1 feature spec; found that F07 was only reachable inside the agent and that LLM retry with backoff was missing; implemented both (`CondenseCommand`, page meter, CLI `condense`, `completeWithRetry`) with tests; drafted this report and the Stage 2 diagrams	Approved the fixes; reviewed this report **[TODO: your edits]**	Commit `83bc4ee`; report draft
7	First runs on real hardware and a real LLM (Nov 12)	Screenshots from the author’s Windows PC: tests passing, then the agent page showing **FAILED · 5 steps** with the model asking “Would you like me to…?”	Diagnosed the stop (the model ended its turn to chat instead of saving); added autonomy rules and bounded nudges, fixed an “Open draft” link to an unsaved draft; added the Maven Wrapper and Windows setup steps	Set up Java, Git and the API key on Windows; ran the tests and the app; ran the agent against Claude Haiku 4.5 and reported the failure with a screenshot; re-ran it after the fix	Second run **COMPLETED** and saved `CV_NorthwindLabs_SoftwareDeveloper` (63% of a page); commit `1251b0c`
8	Readable resume view (Nov 12)	“Why isn’t there a readable LaTeX viewer, like the PDF?”	Built `ResumePreview` (Jake’s-template-like HTML, text escaped against injection)	Noticed the missing preview that Stage 1 promised	Preview on the resume page; commit `aa00500`
9	Real data and calibration (Nov 12)	“We can use my ResumeBase data so there is a pre-existing vault”	Built the JSON backup import; converted the author’s tracker data (65 vault entries, 50 resumes); found that every real one-page resume estimated at 102–113% and recalibrated the page estimate (now 83–98%)	Proposed using his own data; supplied it through his earlier tracker	Commits `17ecbf1`, `4ad069e`
10	Remaining Stage	“What else is left to	Audited every Stage 1 feature spec	Asked for the	Commits `25bf36e`…`724ebae`; 108

	1 gaps (Nov 12)	be implemented?", then "go ahead"	and use case; implemented smart-condense fallback, partial drafts, manual bullet edits, resume-page keyword chips, vault edit/delete/tag filter, editable cover letters with kept failing drafts, "What changed" diff, log-from-resume and follow-ups list, deadlines and editable/manual analysis, goal rules; found and fixed a stale-copy bug in the editor	audit, chose to close the gaps before recording	tests
11	Demonstration videos	Demo script (`docs/stage2/demo-script.md`)	Wrote the click-by-click script per feature	Recorded and narrated the videos; exported the UMLet PNGs (Nov 12)	Section 7

Validation responsibility. Every AI-generated change was built and tested before it was committed (`mvn test`), and the running application was exercised through the GUI and CLI. **[TODO: one sentence on how you reviewed the code, e.g. read the agent loop, tools and state classes and can explain them.]**

6. Development and verification summary

6.1 Weekly commit summary

Each reporting period (Toronto time) has at least five meaningful code commits. All commits are on `main` in the public repository.

Period 1 (2026-10-20 to 2026-10-26): 6 code commits

Commit	Date (Toronto)	Description
`4090a2c`	2026-10-26 19:37	Set up Maven + Spring Boot 3 skeleton with SQLite and git-ignored local key config
`22fe3a7`	2026-10-26 19:39	Add domain model: vault entries, resume blocks, resumes, postings, keywords
`0cba53b`	2026-10-26 19:40	Add SQLite persistence: Repository, SqliteRepository, vault and resume repositories
`e9a1867`	2026-10-26 19:41	Add LLM layer: LLMClient adapter, ClaudeAdapter with tool use, offline mock
`7b832e6`	2026-10-26 19:43	F01: import a Jake's-template LaTeX resume into the library and experience vault
`4ef9eb0`	2026-10-26 19:44	F01 GUI: vault page to import a .tex resume, add entries, search and list the library

Period 2 (2026-10-27 to 2026-11-02): 6 code commits

Commit	Date (Toronto)	Description
`91d30b2`	2026-11-02 21:19	F02: import job postings and analyze them with the LLM (Template Method)
`c0d60a7`	2026-11-02 21:21	F03 + F04: deterministic fit score, closest saved resume, strategy, and ATS keyword scan
`7cdebc6`	2026-11-02 21:23	F05 core: agent reason-act-observe loop with tool registry and live events (Observer)
`7d7e693`	2026-11-02 21:26	F05 tools: find similar resume, search vault, keyword scan, compose and save; LaTeX assembler
`3104d6b`	2026-11-02 21:28	F07: one-page fitting with page estimate and interchangeable condense strategies (Strategy)
`240592e`	2026-11-02 21:29	F06: ask_user tool with vault learning so the agent never asks the same question twice

Period 3 (2026-11-03 to 2026-11-09): 6 code commits

Commit	Date (Toronto)	Description
`670eca2`	2026-11-09 17:06	F11 + F12: application tracking with seven states (State) and an observing dashboard (Observer)
`c701e70`	2026-11-09 17:08	F08: natural-language revision and manual edits with undo/redo (Command)
`f899ecb`	2026-11-09 17:09	F09: cover letter drafting from the tailored resume and letter-safe vault facts (Template Method)
`3696ae7`	2026-11-09 17:10	F10: resume library with use-as-is reuse and export to .tex / cover-letter files
`565700d`	2026-11-09 17:13	CLI (picocli): a command for every feature, –json output, and an interactive shell
`e498dda`	2026-11-09 17:19	Web GUI: postings, live agent page over SSE, resumes, tracker and dashboard (Thymeleaf)

After the reporting periods: `83bc4ee` F07 in the GUI and CLI as an undoable command; agent retries transient LLM errors; `329b94c` Stage 2 report draft, updated UMLet class diagrams, demo script and README; `99bee20` Add Maven Wrapper (mvnw / mvnw.cmd) so the app builds with only Java installed; Windows setup in README; `612cb83` Report: Claude Pro subscription cost in the AI tools table; `beb96b1` Stage 2 class diagrams exported from UMLet; `a1dcf39` Report: include the Stage 2 UMLet class diagrams; `c103c96` Report build: embed diagram images and keep dollar amounts as text; `1251b0c` Agent keeps working when the model stops to chat: autonomy rules, up to 2 nudges, no link to unsaved drafts; `aa00500` Readable resume preview on the resume page (Jake's template layout, rendered in Java); `17ecbf1` F01: import a JSON backup of vault entries and resumes (GUI + CLI vault import-backup); `4ad069e` Calibrate the page estimate on 50 real one-page resumes (63 lines per page); `25bf36e` Agent keeps partial drafts; smart condense falls back to quick condense; condense targets 98%; `bc3bd4c` Resume page: manual bullet edits, keyword chips, What changed diff, log application; follow-ups list; `44f8c31` Vault entries editable and deletable with a tag filter; cover letters editable, failed drafts kept with problems; `724ebae` Posting deadlines and an editable analysis (or manual entry when analysis fails); goals must be >= 1

6.2 Verification performed

- **Automated tests: 108, all passing** (`mvn test`; JUnit 5 + Spring Boot Test + MockMvc; SQLite in memory; the mock LLM replaces Claude):

Area	Test classes (tests)	What they check
Domain and parsing	`DomainModelTest` (5), `LatexResumeParserTest` (7), `ImportResumeTest` (4), `SqliteRepositoryTest` (3), `BackupImporterTest` (3)	keyword word boundaries, chronology, placeholders; both `\resumeSubheading` argument orders, tags, skills, escaped `%`, warnings; idempotent import; round trips; backup kinds/uses, resumes without duplicate vault entries
LLM layer	`ClaudeAdapterTest` (5), `PostingAnalysisTaskTest` (4), `CoverLetterTest` (4), `PostingFlowTest` (2), `AnalysisEditTest` (3)	tool-use JSON translation, merged tool results, retries with feedback, give-up, letters-only vs. context-only facts, failing draft kept with problems, letter edits; deadline extraction, manual analysis
Fit, scan, library, preview	`FitAndKeywordTest` (5), `LibraryExportTest` (5), `AssemblerAndNamingTest` (4), `ResumePreviewTest` (2)	scores and gaps, placeholder resumes never reused, backed/thin/known/missing, reuse naming, path-escape protection, no bold in Education/Skills; preview escapes HTML
Agent	`AgentControllerTest` (8), `TailorAgentTest` (1), `AskUserToolTest` (3), `CondenseAndRetryTest` (3)	full tool loop, invalid/unknown tool calls, step limit, LLM failure and retry, chatty model nudged to save, partial draft kept; end-to-end tailoring with invented numbers rejected; ask once and remember
Edits, condense	`RevisionTest` (5), `ResumeDiffTest` (2), `CondenseStrategyTest` (5)	undo/redo without a second LLM call, redo cleared by new edits, what-changed diff, reload of a replaced resume; rule order, jobs kept, smart plan validated, retried and falling back to

		rules
Tracking	`ApplicationStateTest` (6), `TrackerFlowTest` (4)	legal/illegal transitions, follow-ups, revival, dashboard math, streaks, goal rules, persistence, follow-ups-due list
Interfaces	`CliCommandsTest` (4), `WebPagesTest` (3), `VaultControllerTest` (5), `AgentSessionManagerTest` (2), `CoopPilotApplicationTest` (1)	every CLI feature, JSON output, shell undo; click-through of every page incl. bullet edits, analysis edit, log from resume; vault edit/delete/tag filter/duplicate message; SSE question answered from the browser; friendly errors

- **Packaged application runs:** the built jar was run in CLI mode (import, backup import, library `--json`, log application, dashboard) and in GUI mode with sample data; page screenshots were checked after each GUI change.
- **Author's machine (Windows 11):** `mvnw.cmd test` passed and the GUI ran at http://localhost:8080.
- **Real LLM (Claude Haiku 4.5):** first run FAILED after 5 steps because the model stopped to ask a question in chat (fixed, C20); the second run COMPLETED: find similar resume → keyword scan → three vault searches → compose → keyword scan → estimate page → save, producing a one-page `CV_NorthwindLabs_SoftwareDeveloper`. **[TODO: results of the rest of your test run (condense, revise/undo, cover letter, tracker, dashboard)]**
- **Real data:** the author's 65 vault entries and 50 resumes imported with no parser warnings.

6.3 Important defects found and corrected

Defect	Found by	Fix
"Royal Bank of Canada" named `RBOC` instead of `RBC`	`AssemblerAndNamingTest`	Skip "of/the/and" in acronyms (`ExportService.companyPart`)
Spring could not create `AgentSessionManager` (two constructors)	`WebPagesTest` context load	Marked the injection constructor `@Autowired`
Tracker "Move to" menu listed statuses in random order	Screenshot review of the running app	`allowedNext()` returns lifecycle order (`EnumSet`)
F07 only usable inside the agent; no LLM retry	Design-vs-code review (log #6)	`CondenseCommand`, page meter, CLI `condense`; `completeWithRetry`
Agent stopped to ask "Would you like me to…?" and never saved	First real-LLM run (log #7)	Autonomy rules + bounded nudges (C20)
"Open draft" button linked to a draft that was never stored	First real-LLM run	Link only for stored resumes; partial drafts now stored (C22)
Every real one-page resume estimated over one page (102–113%)	Importing the author's real resumes (log #9)	Recalibrated to 63 lines per page (C19)
Editor kept editing a stale copy after the agent saved a new version under the same name	Code review during the gap work (log #10)	`ResumeEditor` reloads when the stored LaTeX changed
Preview showed "Languages:Java" without a space	Screenshot of the new preview	`ResumePreview.skillLine`

6.4 Remaining limitations

See Section 1: real-model behaviour checked manually only (KUMA evaluation in Stage 3), non-numeric embellishments not blocked deterministically, in-memory undo history, heuristic page estimate, single user, and the one unimplemented Stage 1 item (C07, the second LLM adapter).

7. Demonstration index

Each video shows the feature ID and name, how it is invoked and its input, the running application and its output, where AI is involved, and an error or alternative case. Together they show both the GUI and the CLI. Links are unlisted YouTube videos, viewable without an access request.

Feature	Video	Interface shown	Error / alternative shown
F01 Experience vault and resume import	V01 **[TODO: link]**	GUI (import, backup import, edit, tag filter) + CLI `vault import`	empty upload; duplicate skill refused
F02 Posting import and analysis	V02 **[TODO: link]**	GUI	editing the analysis / filling it in by hand

F03 Fit analysis and strategy	V03 **[TODO: link]**	GUI + CLI `fit`	resume with placeholders never suggested
F04 ATS keyword scan	V04 **[TODO: link]**	GUI chips (posting and resume pages) + CLI `scan`	chips change after an edit
F05 Agentic tailoring	V05 **[TODO: link]**	GUI live steps + CLI `tailor`	a rejected tool call / invented number
F06 Clarifying questions and vault learning	V06 **[TODO: link]**	GUI question box	skipping a question; the answer reused later
F07 One-page fitting	V07 **[TODO: link]**	GUI meter + Quick/Smart condense, CLI `condense`	undo the condense
F08 Revision with undo/redo	V08 **[TODO: link]**	GUI (revise, ✎ bullet edit, What changed) + CLI `shell`	undo with nothing left
F09 Cover letter	V09 **[TODO: link]**	GUI (draft, edit, save, download)	problems shown for a draft that fails validation
F10 Library, reuse and export	V10 **[TODO: link]**	GUI “Use as-is” + download, CLI `export`	reuse refused for a placeholder resume
F11 Application tracking	V11 **[TODO: link]**	GUI (log from resume page, tracker) + CLI `app status`	illegal move (e.g. To apply → Offer) refused
F12 Dashboard and goals	V12 **[TODO: link]**	GUI + CLI `dashboard`	goal of 0 refused; follow-ups due

---

Generated 2026-11-12 from `docs/stage2/report/`.